eBPF Basics

Mainack Mondal
Sandip Chakraborty

CS 60203
Autumn 2026



Outline

- What is eBPF, and why?
  - Motivation, classic BPF vs eBPF, the big picture
- How it works
  - Workflow, virtual machine, verifier and its limits
- Building blocks
  - Program types and return values, networking hooks, maps, helpers, tail calls
- Examples
  - XDP packet filter, tracing with bpftrace
- Toolchain, CO-RE, lifetime and permissions
- eBPF in practice
  - Real-world uses, limitations, eBPF vs kernel modules vs kernel bypass

What is eBPF?

- Run small, **sandboxed programs inside the Linux kernel** when an event happens
- No change to kernel source, no kernel module, no reboot
- A verifier proves the program is safe before it runs; a JIT makes it run at native speed
- Often called **"JavaScript for the kernel"**: the kernel becomes programmable at run time

Name history

BPF = Berkeley Packet Filter (McCanne and Jacobson, 1993), built for tcpdump. Extended BPF arrived in Linux 3.18 (2014). Today "BPF" usually means eBPF.

Networking

XDP, tc, load balancing, Kubernetes networking (Cilium)

Observability

Tracing, profiling, latency histograms (bpftrace, BCC)

Security

Syscall filtering (seccomp), LSM policies, runtime detection

Scheduling and more

TCP congestion control, CPU schedulers (sched_ext)

Why eBPF? Changing Kernel Behaviour Was Hard

Suppose you want to count packets per flow, find slow disk I/O, or drop attack traffic early. Before eBPF, you had three options:

Change the kernel source

- Write a patch and send it upstream
- Review and merge can take months
- Years before distributions ship it

Write a kernel module

- Full power, but no safety net
- One bug can crash or compromise the machine
- Rebuild for every kernel version

Do it in user space

- Copy packets or events up to user space
- Copies and context switches cost CPU
- Little visibility inside the kernel

eBPF: the fourth option

- Load a small program at run time. The kernel checks it, compiles it and runs it exactly where the event happens.
- Safe like user space, fast like kernel code, and no reboot or recompile of the kernel.

Classic BPF (cBPF) vs Extended BPF (eBPF)

Feature	cBPF (1993)	eBPF (2014 onwards)
Registers	2 x 32-bit (A, X)	11 x 64-bit (R0 to R10)
Memory	16-word scratch area	512-byte stack + maps
Jumps	Forward only	Forward and backward (bounded loops)
Kernel services	None	Helpers and kfuncs
State across runs	None	Maps
Attach points	Sockets (tcpdump), seccomp	Network, tracing, security, scheduler
Loading	setsockopt(SO_ATTACH_FILTER)	bpf() system call
Execution	Interpreter (JIT added later)	JIT on all major architectures

See cBPF yourself

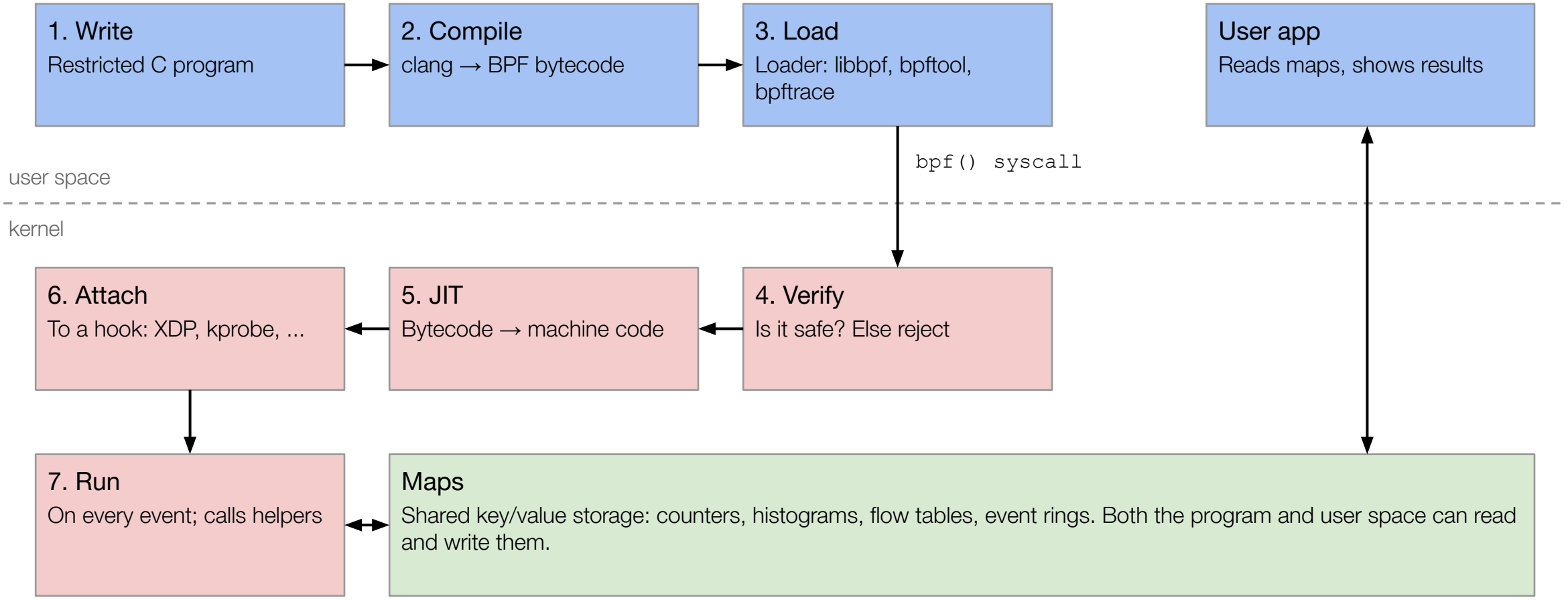
tcpdump compiles its filter to cBPF. Print it with:

```
$ tcpdump -d 'tcp port 80'
```

cBPF is still around

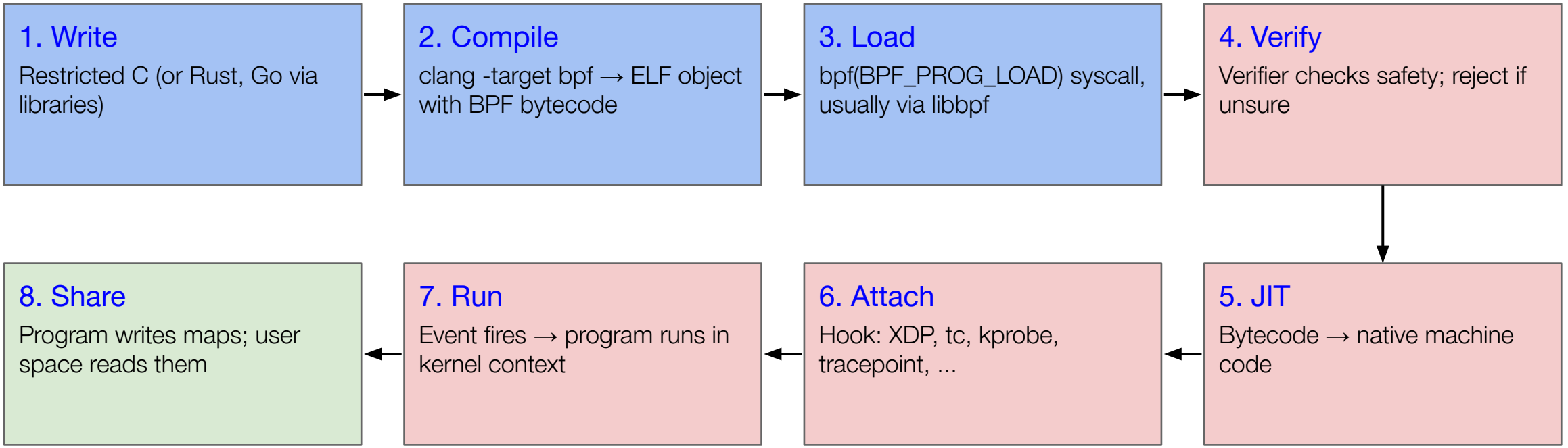
- The kernel translates cBPF into eBPF internally
- seccomp filters are still written in cBPF
- "BPF" today almost always means eBPF

The Big Picture: Who Does What



blue: user space pink: kernel green: shared

How It Works: From C Code to a Running Program



blue: user space *pink: kernel* *green: shared*

```
$ clang -O2 -g -target bpf -c prog.bpf.c -o prog.bpf.o
$ bpftool prog load prog.bpf.o /sys/fs/bpf/prog      # load and pin
$ bpftool prog show                                  # list loaded programs
```

The eBPF Virtual Machine

Register	Role
R0	Return value of the program and of helper calls
R1 to R5	Arguments to helpers; R1 holds the context pointer at entry
R6 to R9	Callee-saved, keep values across helper calls
R10	Read-only frame pointer to the 512-byte stack

Instruction encoding (64 bits)



A 64-bit constant load uses two slots (16 bytes).

- Small 64-bit RISC instruction set: ALU, load/store, jumps, call, exit
- Registers map one-to-one onto real CPU registers, so the JIT is simple and fast
- **512-byte stack** per program; larger data goes in maps
- Program receives a **context**: e.g. xdp_md for XDP, pt_regs for kprobes
- Inspect bytecode: llvm-objdump -d prog.bpf.o

The Verifier: Safety Before Speed

- Runs at load time and **walks every possible path** of the program
- Tracks the type and value range of every register
- Rejects the program unless it can prove:
  - it always terminates (no unbounded loops)
  - every memory access is in bounds
  - pointers are null-checked before use
  - no uninitialised reads, no kernel pointer leaks
  - only helpers allowed for this program type are called
- Limit: about 1 million instructions explored per program

Rejected: no bounds check

```
struct ethhdr *eth = data;
if (eth->h_proto == ...)    /* may read
                             past data_end */
```

Accepted: bounds proved first

```
struct ethhdr *eth = data;
if ((void *)(eth + 1) > data_end)
    return XDP_PASS;
if (eth->h_proto == ...)    /* safe */
```

When the verifier says no, the error log shows the exact instruction and register state. Reading these logs is a key eBPF skill.

Verifier Limits and How to Work with Them

Limit	Value
Instructions explored	1 million per program (since 5.2); 4,096 for unprivileged users
Stack size	512 bytes per program
Loops	Must be provably bounded (bounded loops since 5.3; bpf_loop() helper since 5.17)
Tail calls	Chain of at most 33 programs
Function arguments	At most 5, passed in R1 to R5
Memory access	Only inside checked packet, map value, stack or context bounds

Tips to pass the verifier

- Check every pointer: bounds for packets, NULL for map lookups
- Give loops a small, fixed upper bound
- Keep big buffers in a per-CPU array map, not on the stack
- Split large logic with BPF-to-BPF calls or tail calls

Read the log

The verifier log names the failing instruction and the register state

Common error messages to recognise

```
"invalid access to packet"   "R1 invalid mem access"   "unbounded memory access"   "back-edge from insn"
```

Where Programs Attach: Hooks and Program Types

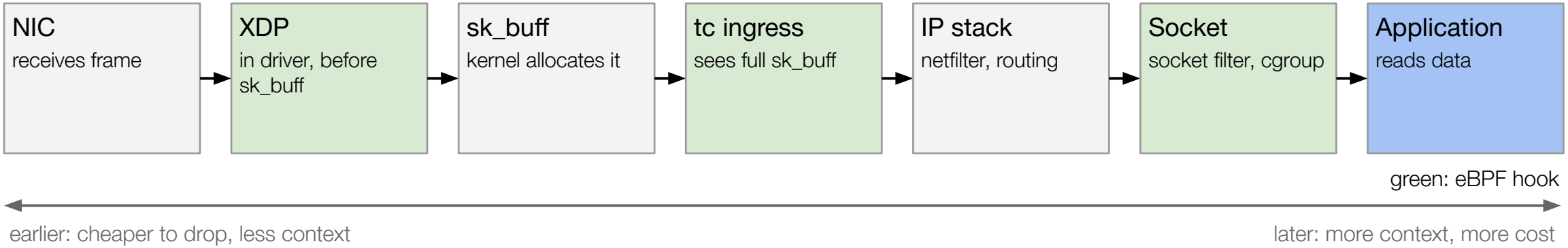
Program type	Attach point	Typical use
XDP	NIC driver RX, before sk_buff	DDoS drop, load balancing, fast forwarding
tc (cls_bpf)	Traffic control ingress / egress, has sk_buff	Policy, NAT, container networking
Socket filter / sockops	Sockets and TCP events	Filtering, tuning, redirect between sockets
cgroup hooks	connect, bind, send per cgroup	Per-container network policy
kprobe / kretprobe	Entry or return of almost any kernel function	Dynamic tracing (not stable across versions)
Tracepoints	Static points placed by kernel developers	Stable tracing: syscalls, scheduler, block I/O
fentry / fexit	Function entry / exit using BTF	Faster, typed tracing
uprobes, USDT	Functions and probes in user programs	Tracing apps, libraries, runtimes
perf_event	Timers and hardware counters	CPU profiling, flame graphs
LSM, struct_ops	Security hooks; kernel ops tables	Security policy, TCP CC, sched_ext

Context In, Verdict Out

Program type	Input (context in R1)	Return value (R0) means
XDP	struct xdp_md: raw packet bytes	XDP_PASS, XDP_DROP, XDP_TX (send back out), XDP_REDIRECT, XDP_ABORTED
tc	struct __sk_buff	TC_ACT_OK (continue), TC_ACT_SHOT (drop), TC_ACT_REDIRECT
Socket filter	struct __sk_buff	Number of bytes to keep; 0 drops the packet for this socket
cgroup (connect, bind)	struct bpf_sock_addr	1 allows the call, 0 rejects it
kprobe, tracepoint	Registers or tracepoint fields	Mostly ignored; results go to maps
LSM	Arguments of the security hook	0 allows; a negative error such as -EPERM denies

Same pattern everywhere: the kernel passes a **context pointer** in R1, runs the program, and reads the **verdict** from R0. For tracing, the useful output is what the program writes into maps.

Where the Networking Hooks Sit



XDP native mode

- Runs inside the NIC driver
- Fastest; needs driver support (most 10G+ NICs)

XDP generic mode

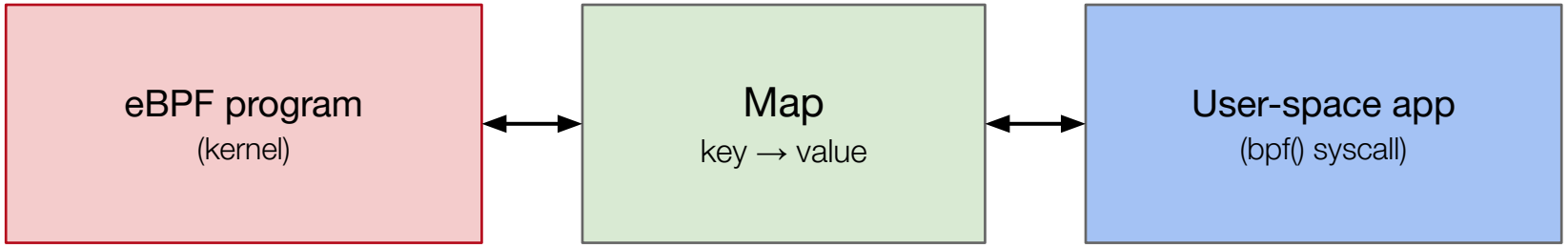
- Runs after sk_buff is built
- Works on any NIC; slower; good for testing

XDP offload mode

- Runs on a SmartNIC itself
- Frees host CPU; very few NICs support it

Egress path: application → socket → IP stack → tc egress (eBPF) → driver → NIC. XDP is on the receive path only, but XDP_REDIRECT can send a frame out of another interface.

Maps: State Shared Between Kernel and User Space



Map type	Use
HASH, ARRAY	General counters and tables
PERCPU_HASH / ARRAY	One copy per CPU: no atomics
LRU_HASH	Flow tables that evict old entries
LPM_TRIE	Longest-prefix match (routing, ACLs)
RINGBUF	Stream events to user space (5.8+)
PROG_ARRAY	Table of programs for tail calls
DEVMAP, CPUMAP, XSKMAP	XDP redirect targets (NIC, CPU, AF_XDP)

```
struct {
 __uint(type,
   BPF_MAP_TYPE_HASH);
 __uint(max_entries,
   1024);
 __type(key, __u32);
 __type(value, __u64);
} cnt SEC(".maps");
```

- Kernel side: bpf_map_lookup_elem(), bpf_map_update_elem()
- User side: same operations through the bpf() syscall or libbpf
- Maps can be pinned in /sys/fs/bpf and outlive the loader

Getting Data Out to User Space

1. Poll a map

- Program updates counters or histograms in a map
- User space reads them every few seconds
- Very cheap; ideal for aggregates
- Individual events are not kept

2. Perf buffer

- One buffer per CPU
- Program sends a record for each event
- Events from different CPUs may arrive out of order
- Memory is wasted when load is uneven

3. Ring buffer (5.8+)

- One buffer shared by all CPUs
- Keeps the order of events
- Reserve, fill, submit: no extra copy
- Preferred choice for new code

Rule of thumb

- Aggregate inside the kernel whenever you can. Sending every event to user space is the main cost of most eBPF tools.
- Per-CPU maps avoid locks and atomics; user space adds up the per-CPU values when it reads.

Helpers, kfuncs and Tail Calls

Helper functions

- A fixed, stable API the kernel offers to BPF programs
- Which helpers are allowed depends on the program type
- Examples:
  - `bpf_map_lookup_elem()`, `bpf_ktime_get_ns()`
  - `bpf_get_current_pid_tgid()`, `bpf_redirect()`

kfuncs

- Kernel functions exposed directly to BPF
- More flexible than helpers, but no stability promise across kernel versions
- Most new kernel features for BPF now come as kfuncs

Tail calls

- Jump to another program through a PROG_ARRAY map; does not return
- Chains up to 33 programs; used to split big logic or swap stages at run time

BPF-to-BPF calls

- Normal function calls inside one program, with their own stack frames
- Keep code modular while the verifier still checks everything

Example: Drop UDP Packets to Port 9999 with XDP

```
SEC("xdp")
int xdp_drop_udp(struct xdp_md *ctx)
{
  void *data     = (void *)(long)ctx->data;
  void *data_end = (void *)(long)ctx->data_end;

  struct ethhdr *eth = data;
  if ((void *)(eth + 1) > data_end) return XDP_PASS;
  if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;

  struct iphdr *ip = (void *)(eth + 1);
  if ((void *)(ip + 1) > data_end) return XDP_PASS;
  if (ip->protocol != IPPROTO_UDP) return XDP_PASS;

  struct udphdr *udp = (void *)ip + ip->ihl * 4;
  if ((void *)(udp + 1) > data_end) return XDP_PASS;
  if (udp->dest != bpf_htons(9999)) return XDP_PASS;

  __u32 k = 0;
  __u64 *n = bpf_map_lookup_elem(&drops, &k);
  if (n) (*n)++;            /* per-CPU counter */
  return XDP_DROP;
}
char LICENSE[] SEC("license") = "GPL";
```

- Every pointer is checked against **data_end** before use
- **drops** is a PERCPU_ARRAY map with one entry
- Return value decides the packet's fate
- Headers needed: linux/bpf.h, if_ether.h, ip.h, udp.h, in.h, bpf/bpf_helpers.h, bpf/bpf_endian.h

```
$ clang -O2 -g -target bpf \
    -c xdp_drop.c -o xdp_drop.o
$ ip link set dev eth0 \
    xdp obj xdp_drop.o sec xdp
$ bpftool map dump name drops
$ ip link set dev eth0 xdp off
```

Tracing with bpftrace

```
probe /filter/ { action }
```

- **Probe:** where to attach, e.g. kprobe:, kretprobe:, tracepoint:, uprobe:, profile:
- **Filter:** optional condition, e.g. /pid == 1234/
- **Action:** what to do; often update a map named @something
- **Built-ins:** pid, tid, comm, nsecs, retval, kstack
- **Aggregations:** count(), sum(), avg(), hist()

Which files are opened, and by whom?

```
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename)); }'
```

How many system calls does each process make?

```
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
```

What sizes do read() calls return?

```
bpftrace -e 'kretprobe:vfs_read { @bytes = hist(retval); }'
```

bpftrace turns the script into eBPF, loads it, and prints all maps when you press Ctrl-C. Use it for quick questions; use libbpf for long-running tools.

Toolchain and CO-RE (Compile Once, Run Everywhere)

bpftrace: High-level tracing language; one-liners and short scripts

BCC: Python/Lua front end, compiles at run time; large tool collection

libbpf (C): Standard loader library; used with clang and bpftool

cilium/ebpf, Aya: Go and Rust libraries for production systems

bpftool: Inspect and manage programs, maps, BTF; generate skeletons

The problem CO-RE solves

- Kernel structs (e.g. task_struct) change layout between versions
- **BTF** (BPF Type Format) ships type info with the kernel: /sys/kernel/btf/vmlinux
- libbpf **relocates field offsets at load time**, so one binary runs on many kernels

```
# all kernel types in one header
$ bpftool btf dump file /sys/kernel/btf/vmlinux \
      format c > vmlinux.h
/* in BPF code: portable field read */
pid = BPF_CORE_READ(task, real_parent, tgid);
```

Lifetime and Permissions

How long does a program live?

- Programs, maps and links are kernel objects with reference counts
- The loader holds file descriptors; when the last one closes, the object is freed
- Pin to /sys/fs/bpf to keep it alive after the loader exits
- A bpf_link (5.7+) ties a program to a hook; drop the link to detach cleanly

Who can load eBPF?

- Most program types need root or specific capabilities
- CAP_BPF (5.8+) to load; plus CAP_PERFMON for tracing or CAP_NET_ADMIN for networking
- Most distributions disable unprivileged eBPF (kernel.unprivileged_bpf_disabled)
- Containers cannot load eBPF unless given these capabilities

See what is loaded on a machine

```
$ bpftool prog list     $ bpftool map list     $ bpftool link list
```

eBPF in the Real World

Cilium

- Kubernetes networking, policy and observability
- Can replace iptables-based kube-proxy

Katran (Meta)

- Open-source layer 4 load balancer
- Built on XDP

DDoS mitigation

- Cloudflare and others drop attack traffic with XDP
- Before the network stack spends CPU on it

Observability

- bpftrace and BCC tools for performance work
- Pixie, Grafana Beyla: tracing without code changes

Runtime security

- Falco, Tetragon watch syscalls and kernel events
- Can block actions through LSM hooks

Beyond servers

- Android: per-app network accounting
- sched_ext (Linux 6.12), eBPF for Windows

blue: networking green: observability pink: security yellow: other

Limitations and Pitfalls

Programming limits

- Restricted C: bounded loops, no floating point, small stack
- The verifier may reject code that is actually correct
- Debugging is limited: bpf_printk() and verifier logs
- Linux first; the Windows port covers only a subset

Operational concerns

- Available features depend on the kernel version
- kprobes on internal functions can break after an upgrade
- Heavy probes on hot paths add real overhead
- Attackers use eBPF too (rootkits): control who can load it

Good practice

Prefer stable hooks (tracepoints, fentry with BTF), aggregate inside the kernel, and test on the kernels you will deploy to.

eBPF vs Kernel Modules vs Kernel Bypass

Aspect	Kernel module	eBPF	Kernel bypass (DPDK)
Where code runs	Kernel	Kernel, sandboxed	User space
Safety	None: a bug can crash the kernel	Verifier-checked before running	Process isolation + IOMMU
Deployment	Build per kernel version	Load at run time, no reboot	NIC unbound from kernel
Kernel features	Full access	Keeps stack, tools, policies	Lost on that NIC
Flexibility	Anything	Restricted by verifier and helpers	Anything in user space
Packet speed	Kernel stack speed	High with XDP	Highest (line rate)
Typical use	Drivers, file systems	Tracing, XDP, security, policy	NFV, telco, appliances

eBPF is the middle path: much of the speed of bypass, with the safety and tooling of the kernel.

Summary

- eBPF runs small, safe programs inside the kernel when events happen, with no reboot or module
- It grew out of classic BPF: more registers, a stack, maps, helpers and many more hooks
- Pipeline: compile to bytecode → load → verify → JIT → attach to a hook
- The verifier makes it safe: bounded, in-bounds, null-checked, allowed helpers only
- Hooks cover networking (XDP, tc), tracing (kprobes, tracepoints) and security (LSM); the return value is the verdict
- Maps keep state and connect kernel programs with user space; ring buffers stream events
- Tools: bpftrace and BCC for tracing; libbpf, Go and Rust libraries for production; CO-RE for portability
- A middle path between kernel modules and full kernel bypass, with real limits to respect

References

- ebpf.io: What is eBPF?
- Linux kernel BPF documentation
- S. McCanne, V. Jacobson, The BSD Packet Filter, USENIX Winter 1993
- T. Høiland-Jørgensen et al., The eXpress Data Path, CoNEXT 2018
- B. Gregg, BPF Performance Tools, Addison-Wesley 2019
- libbpf-bootstrap: starter templates for libbpf + CO-RE
- bpftrace reference guide
- A. Nakryiko, BPF CO-RE reference guide
- XDP tutorial (xdp-project)
- eBPF Foundation: ebpf.foundation
- Kernel documentation: BPF ring buffer (Documentation/bpf/ringbuf.rst)